Design Rationale

1. Overview

The CodeWiz AI Assistant is a modular retrieval-augmented generation (RAG) system designed to answer questions over a dynamic and growing collection of business documents. It integrates Google Drive-based ingestion, semantic chunking, vector retrieval, and LLM-based reasoning in a maintainable and transparent architecture.

Key design principles:

- Grounded answers: Responses are strictly based on company documentation; no hallucinations.

- Modularity: Decoupled sync, preprocessing, embedding, retrieval, and answering.

- Local-first: Minimal external dependencies for easy prototyping and privacy.

- Extensibility: Ready for future cloud migration, new personas, and workflow expansions.

2. Architecture Components

2.1 Google Drive Ingestion & Sync

A central folder on Google Drive is the source of truth. A Google Apps Script runs weekly to scan for new or modified files, converting Google Docs, Sheets, and Slides to PDF for standardization. File timestamps are tracked to avoid redundant downloads. Downloads are currently handled manually (e.g., via gdown) to keep the system simple and avoid cloud API credentials. (can be automated with google drive api)

2.2 Preprocessing & Bucketing

The `preprocess.py` module handles document cleaning, metadata extraction, and chunking with overlap to maintain context. It applies semantic bucketing (topic classification) using keyword and pattern matching, and redacts sensitive information (emails, phone numbers, names, IDs). Deduplication removes redundant or low-confidence chunks.

2.3 Embedding Generator

Embeddings are created with OpenAI's `text-embedding-3-large` model, providing semantically rich vector representations for each chunk.

2.4 Vector Database

A local FAISS index stores these embeddings, supporting fast similarity search. This is ideal for MVP testing and can be migrated to a managed vector DB (e.g., Weaviate) for scale.

2.5 LLM-Based Answering

User queries are passed to a retrieval chain that uses the vector database to fetch relevant chunks. The retrieved context is combined with the user's question and sent to GPT-4 Turbo for answer generation. Prompts are persona-aware and enforce strict context usage. If no answer is found, the system replies:

"I couldn't find a clear answer based on current documentation."

2.6 Logging & Analytics

All user questions, answers, and personas are logged to a CSV file. This data powers a Streamlit dashboard that:

- Shows the most common questions by persona

- Lists unanswered questions (where the fallback message was returned)

- Helps refine prompt design and chunking strategies over time

2.7 User Interface

A Streamlit web interface allows persona selection, question input, and answer display. Internal users can view which document chunks and sources were used for each answer.

3. Design Trade-offs

-Apps Script vs. Drive API: Chosen for simplicity, despite less fine-grained control.

- Chunking vs. Document-level embeddings: Chunking prioritized for more relevant and precise answers.

- PDF conversion: Standardizing to PDF simplifies preprocessing.

PDF was chosen as the standardized format because it preserves layout consistency across different document types, avoiding formatting errors that can happen with DOCX or HTML conversions. In addition, PDFs are broadly compatible with text extraction frameworks like LangChain and Unstructured, supporting multi-page parsing, OCR, and metadata handling in a stable way. Google Apps Script can export Google Docs, Sheets, and Slides to PDF easily without requiring extra conversion libraries. Since the company's document base includes a wide variety of formats, using PDF ensures uniform, predictable downstream processing while still keeping the option to store non-PDF files in raw form if needed.

- GPT-4 Turbo vs. local LLM: Vendor lock-in accepted for higher answer quality, with a path to swap later if needed.

4. Future Improvements

- Automate download from Drive (replace gdown)

- Multi-LLM fallback to reduce cost and latency

- Improved classification of lead types and personas

- Enhanced question analytics with live dashboards

- JSON-structured responses for future integrations

- Hybrid search (vector + keyword)

- Chat interface with history/memory

5. Cost Considerations

- Based on current OpenAI costs:

~$0.0097 per question

Projected ~$29/month for 100 daily queries

This is sustainable under the current budget for a prototype.

6. Conclusion

This architecture meets the CodeWiz candidate case brief with a grounded, local-first, and modular design. The system is ready to handle queries over an evolving document base while respecting user personas, with a clear roadmap for future improvements, better persona-based transparency, and advanced analytics via logging and dashboards.